

INTELLIMAIL

AI INBOX MANAGEMENT WEB APP

Yi Zheng Tang

G00425075



Problem & Solution



Problem

- Multiple email and messaging accounts.
- Cluttered and overwhelmed inboxes.
- Leads to excessive time spent on digital communication platforms
- Reduced productivity, missed deadlines, stress-inducing and quicker burnout.



Solution

- A web app that unifies all platforms to a single dashboard.
- Intelligently summarize, categorize, prioritize and automates message management by implementing LLMs, AI Agents and RAG.

Background Research



Problem research

Thursday 11 December 2025 18:23

Before I start the project, I did research on related topics for backing up the problem that my project aims to solve, here is what I found:

Problem research:

Shorter Attention Time span

I found a Q&A article with Dr. Gloria Mark, she stated that attention spans on screens have drastically declined from 2.5 minutes in 2004 to just 44 - 50 seconds by 2021. In current digital era, where people spend most of their time on electronic devices, whether it's for entertainment, working etc.

Another research and practice on personal productivity I found, done by Jill Duffy, stated that when workers switch between tasks, it takes an average of 25 minutes to return to their original task, people also typically encountering an average of 2.3 other tasks before resuming their initial work.

My thought:

I think this is the really common yet serious issue in our current society, it isn't just about shorter focus periods, it also reflects the **cognitive burden of constant switching contexts across multiple communication platforms**. When a user juggles between different applications, it forces their brain to **continuously reload mental context, process different communication styles and different interfaces, this will not just result in time consuming, and also will heavily affects working productivity**.

Therefore, by developing a web app that dedicates to **unify all platforms into a single dashboard, with the same UI design**. Not only will **improve one's productivity, but also reduces the mental effort of adaptation**.

Reference:
<https://annieduke.substack.com/p/q-and-a-with-gloria-mark-author-of>

Excessive Time spent on Communication Platform

I found an economic story blog discussing about **How Digital platforms are making us less productive and more overwhelmed**. While digital platforms like email, Microsoft Teams, Slack, have increased productivity in some ways, they also mean more time responding to messages and less time accomplishing work tasks.

Overwhelmed inboxes, constantly juggling conversations, can create a lot of working anxiety and heavily affect one's working productivity and working quality.

Another article states that email overload occurs when the number of emails that an individual receives exceeds their ability to process and respond to them in a timely and effective manner. **The consequences of email overload also include missed deadlines, ignored messages, unread emails.**

Email overload can also create security risks for businesses due to the sheer volume of messages coming into inboxes each day, **making it difficult for employees to detect malicious content such as phishing scams or malware-infected attachments before they even detect or open them.**

My thought:

This is what we already know, and it is the most common issue we have nowadays, especially after the increase of remote work lately, which makes work/companies more relies on digital platform communication like sending emails, texts. Email overload would be the very first issue they will face. For my personal experience, during my work placement

Solution research

Wednesday, 19 November, 2025 1:21 PM

After understanding the problem, and have a rough idea of the solution. In this section, I will do research to validate the technical approaches and implementations:

1. Technical approaches that will successfully reduce the time spent on email management.
2. Design principles that enable users to process information efficiently across unified platforms.
3. How intelligent automation and AI Agent workflow can further elevates the system in terms of enhance productivity, improve user experience and reduce cognitive load.
4. Implementing frameworks like RAG (Retrieval-Augmented Generation) that enhances LLM accuracy.

Technical approaches that will successfully reduce the time spent on email management

Research shown that AI can increase productivity by 20-30% through automation and optimization of various working processes, which including email management. On top of current existing email sorting algorithms, like sort by label, sort by keywords, or sort by subjects, **AI-driven categorization utilizes machine learning algorithms and natural language processing techniques to automatically classify inboxes into relevant categories, for example work-related messages, personal communication, promotional or advertising emails, spams, and high priority messages.**

<https://gmellius.com/blog/harnessing-ai-for-email-management>

Design principles that enable users to process information efficiently across unified platforms.

Unified communications is defined as a business and marketing concept that integrates multiple enterprise communication services, providing a consistent unified user interface and user experience across multiple devices and media types. The unification of all communication applications and devices into a single platform provides the mobility, presence, and contact capabilities that extend beyond individual channels to all devices a person may use. Research also shown that unified interface provides a consistent and accessible user experience.

Unified chat-driven interfaces also provide a single, conversational entry point that handles all user interactions, which also provides a much better way for users to deal with complex systems.

Therefore, **building a single web interface with React that consolidates Gmail, Outlook, WhatsApp, Instagram, and Teams, with consistent UI design, will reduce the cognitive burden of the user, and provides better productivity and simplicity.**

https://en.wikipedia.org/wiki/Unified_communications
<https://medium.com/@lordNeic/revolutionizing-user-interfaces-my-vision-for-a-unified-chat-driven-ucd-experience-d2e1b14214d3>

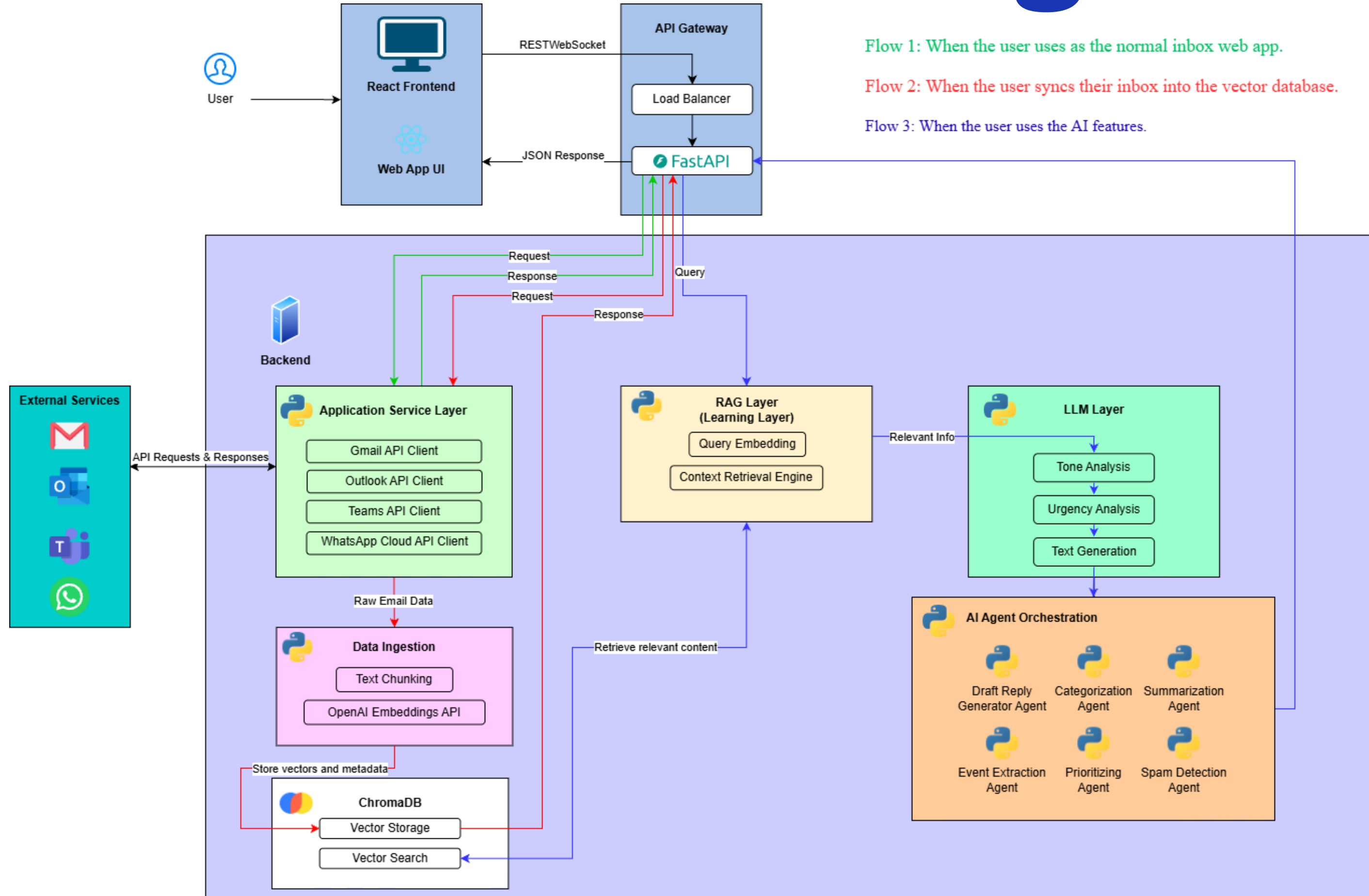
How intelligent automation and AI Agent workflow can further elevates the system in terms of enhance productivity, improve user experience and reduce cognitive load

AI agents are designed to support people, by reducing repetitive work, improving accuracy, and help make faster decisions. Research also shown that organizations that adopt AI agents into their system earns their employee over 2 extra hours daily, which leads to higher performance and working efficiency.

<https://www.glean.com/perspectives/ai-agents-for-repetitive-low-value-tasks>

Implementing frameworks like RAG (Retrieval-Augmented Generation) that enhances LLM accuracy

Architecture Diagram

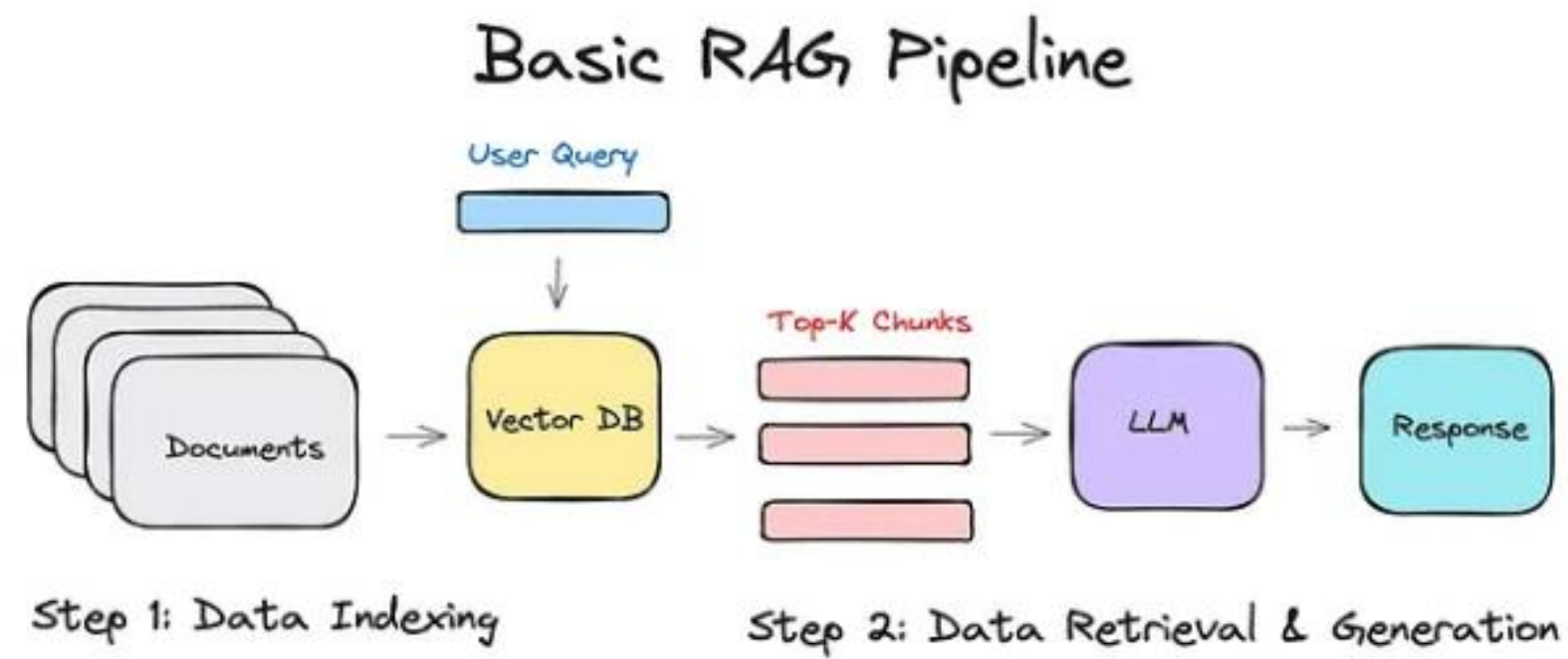




AI Features

RAG (Retrieval-Augmented Generation)

- Framework that connects the pretrained LLM model with given knowledge base.
- In this case, the knowledge base would be the user's emails and inboxes.
- Enhances better accuracy of outputs, prevents hallucinations, irrelevant outputs.



```
def retrieve_context(query_text: str, k: int = 3, where: dict | None = None) -> str:
    if not query_text.strip():
        return ""

    query_embedding = embed_text(query_text)

    try:
        results = query_similar(query_embedding, k=k, where=where)
    except Exception:
        #ChromaDB raises if n_results exceeds the number of stored documents.
        #Return empty string so agents degrade gracefully rather than crashing.
        return ""

    #ChromaDB returns results nested in a list – one inner list per query embedding.
    #Since we always query with exactly one embedding, results are always at index [0].
    documents = results.get("documents", [[]])[0]
    metadatas = results.get("metadatas", [[]])[0]

    if not documents:
        return ""

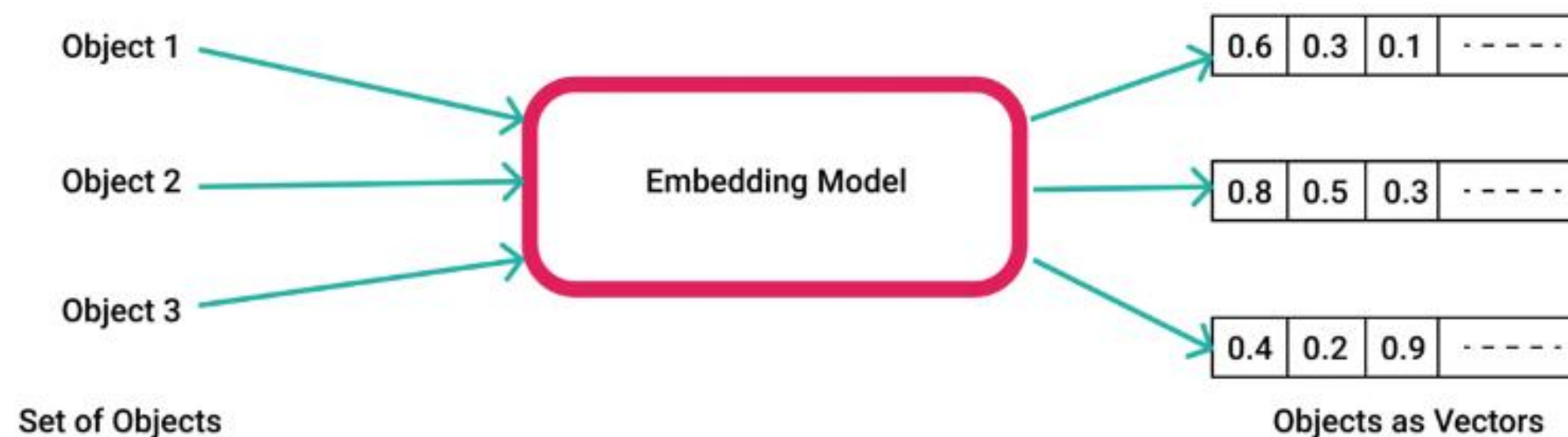
    #Build one formatted section per retrieved email.
    #zip() pairs each document with its metadata dict at the same position.
    #enumerate(start=1) gives a 1-based counter for the section headers.
    sections = []
    for i, (doc, meta) in enumerate(zip(documents, metadatas), start=1):
        sender = meta.get("from", "unknown")
        date = meta.get("date", "unknown")
        subject = meta.get("subject", "")

        header = f"--- Related email {i} ---"
        info = f"From: {sender} | Date: {date} | Subject: {subject}"
        sections.append(f"{header}\n{info}\n{doc}")

    return "\n\n".join(sections)
```


Vector Embeddings

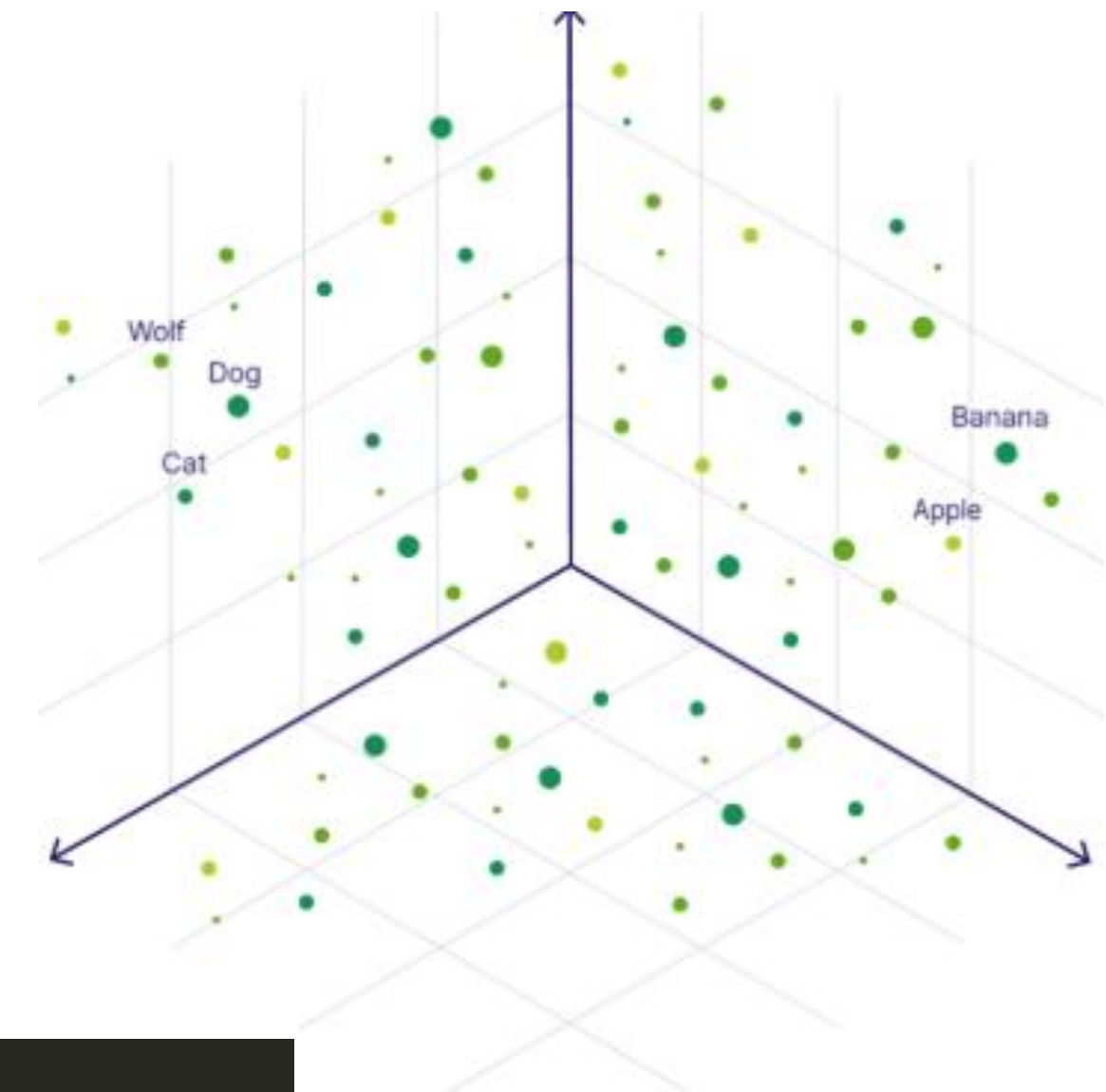
- A way to convert unstructured data (text) into a list of float numbers (vector).
- Using pre-trained embedding model, it captures the meaning of the text, and not just the words.



```
#Calls the OpenAI embedding API and returns a list of 1536 floats.  
#The returned vector encodes the semantic meaning of the text as coordinates in 1536-dimensional space.  
def embed_text(text: str) -> list[float]:  
    response = _client.embeddings.create(  
        model="text-embedding-3-small",  
        input=text  
    )  
    return response.data[0].embedding
```

Semantic Search

- Computers cannot compare meaning through float numbers, but they can calculate the distance between number arrays.
- Similar meaning = vectors that are closer together in space.



```
#Returns the k most semantically similar stored emails to a query embedding.
#Pass a where dict to restrict results by provider, message_type, or any other metadata field.
def query_similar(query_embedding: list[float], k: int = 3, where: dict | None = None) -> dict:
    collection = get_collection()
    params = {
        "query_embeddings": [query_embedding],
        "n_results": k,
        "include": ["documents", "metadatas", "distances"]
    }
    if where:
        params["where"] = where
    return collection.query(**params)
```

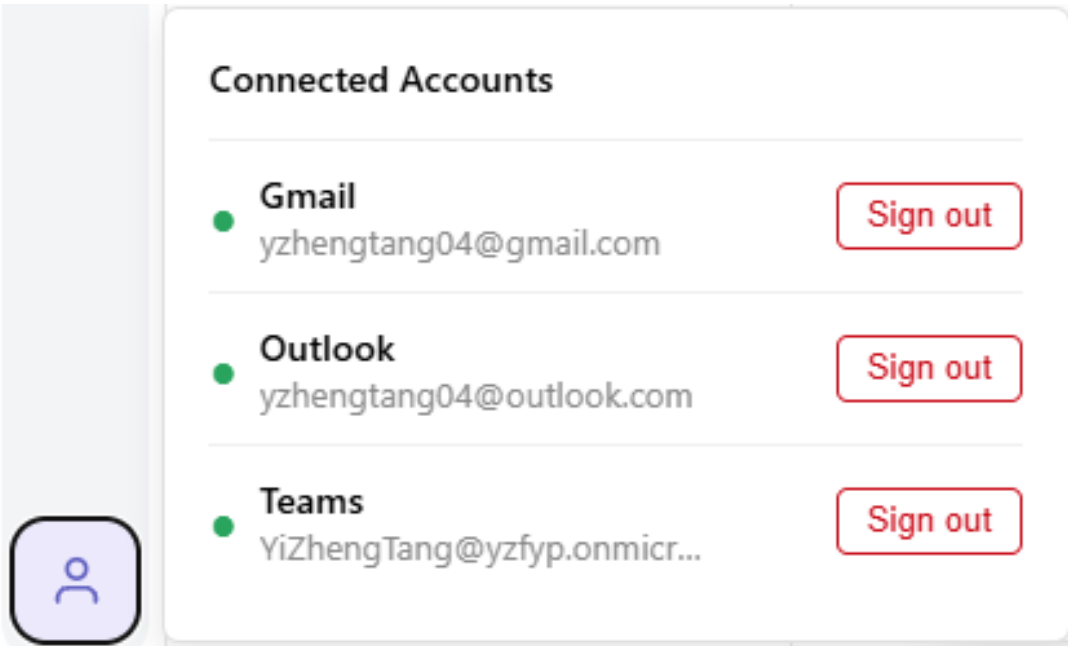



AI Agents

- A function that sends a prompt to an LLM and returns a structured result.
- First, write a concise prompt of what the agent should do, then call the LLM, and then lastly parse the output.
- system prompt: sets the behaviour and rules for the AI.
- user prompt: the actual input for this specific request

```
def _build_categorise_prompt(email_text: str) -> str:  
    return f"Categorise the following email.\n\n<email>\n{email_text}\n</email>"
```

```
def categorize_email(email_text: str) -> str:  
    response = _client.chat.completions.create(  
        model="gpt-4o-mini",  
        messages=[  
            {"role": "system", "content": "You are an email categorisation assistant. Categorise the email into exactly one of the following categories: work, personal, or unknown."},  
            {"role": "user", "content": _build_categorise_prompt(email_text)}  
        ],  
        temperature=0,  
        max_tokens=10  
    )  
    return response.choices[0].message.content.strip().lower()
```



Gmail

SENT

INBOX360

TRASH4

DRAFT

STARRED

> Course

Personal

test



CHAT AI

1. Tamer Zraiq addressed an employee's recent absences from work, noting a specific incident related to a forgotten item and emphasizing the importance of mental health support.
2. Google notified the user of a new sign-in to their account on a Windows device, advising them to secure their account if the sign-in was not authorized.
3. David Jayakumar submitted his code for review, indicating it is attached to the email.
4. Microsoft confirmed an order placed on April 10, 2026, and provided a link to manage subscriptions in the Microsoft 365 admin center.
5. Microsoft informed the user that new apps have connected to their account, advising them to remove any unauthorized access.
6. Microsoft announced the availability of a new trial subscription for Microsoft 365 Business Basic, encouraging the user to sign in and start using it.
7. Uber Eats offered a promotion of 40% off the next ride, encouraging timely transportation.
8. Uber Eats provided an 80% discount for first-time orders, aimed at attracting new customers.
9. Uber Eats reminded users that a 40% discount on rides is about to expire, creating urgency for the offer.
10. Microsoft sent a single-use code to the user for their account, advising them to keep it confidential and only use it on official platforms.

Give me a summary of non spam emails

Gmail

SENT

INBOX360

TRASH4

DRAFT

STARRED

> Course

Personal

test



April 2026

<

>

Mon	Tue	Wed	Thu	Fri	Sat	Sun
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28 14:00 Project discussion at school	29	30			

Testing

- Backend services like Gmail API, Outlook API, database are tested using pytest.
- Using MagicMock, I am able to replace live services like Gmail, Outlook while testing.
- Able to achieve **87%** coverage for Gmail API, **79%** for Outlook API.
- Unable to get more coverage because some functions required real OAuth tokens, which I cannot mock or be faked in unit tests.

```
def test_get_email_messages_returns_formatted_list():
    msg_stub = {'id': 'msg001'}
    batch_resp = {
        'msg001': {
            'threadId': 'thread001',
            'snippet': 'Email snippet',
            'labelIds': ['INBOX', 'UNREAD'],
            'payload': {
                'headers': _make_headers(),
                'parts': []
            }
        }
    }
    service = _make_batch_service('inbox', [msg_stub], batch_resp)
    result = get_email_messages(service, folder_name='inbox', max_results=1)
    assert len(result) == 1
    assert result[0]['id'] == 'msg001'
    assert result[0]['subject'] == 'Test Subject'
    assert result[0]['is_read'] is False
    assert result[0]['thread_id'] == 'thread001'
```

Name	Stmts	Miss	Cover	Missing
Gmail\Google_API.py	47	47	0%	2-98
Gmail\gmail_api.py	517	67	87%	35-36, 77, 128, 171, 296-306, 349, 357-364, 383, 395-405, 420-453, 457-460, 563, 636, 640, 829-837, 900, 937
Gmail\tests__init__.py	2	2	0%	1-2
Gmail\tests\test_delete_emails.py	15	15	0%	1-20
Gmail\tests\test_download_email.py	7	7	0%	1-10
Gmail\tests\test_draft_email.py	11	11	0%	1-27
Gmail\tests\test_fetch_conversation.py	13	13	0%	1-16
Gmail\tests\test_gmail_api.py	18	18	0%	1-24
Gmail\tests\test_labels.py	24	24	0%	1-29
Gmail\tests\test_reply_email.py	20	20	0%	1-32
Gmail\tests\test_search_emails.py	19	19	0%	1-23
Gmail\tests\test_send_email.py	14	14	0%	1-17
Outlook\MS_Graph.py	49	49	0%	2-98
Outlook\outlook_api.py	480	101	79%	108-110, 155, 233-274, 297-337, 354, 400, 407-463, 504-505, 534-535, 542, 627-628, 660, 684, 776-777, 809-810
Outlook\tests__init__.py	2	2	0%	1-2
Outlook\tests\test.py	22	22	0%	1-35
Outlook\tests\test_download_email.py	6	6	0%	1-9
Outlook\tests\test_draft_email.py	48	48	0%	1-88
Outlook\tests\test_fetch_conversation.py	22	22	0%	1-29
Outlook\tests\test_folders.py	91	91	0%	1-127



Timeline



Nov – Jan

Microservices API
Integration and
finishing backend Dev.



Jan – Feb

Research into AI, RAG,
vector embeddings.



Feb – Apr

Implement AI layer,
frontend design.



Mar – May

Finishing Frontend
Dev, Software Testing,
Documentation and
Report Writing.

Reflection & Notes

YI ZHENG TANG - STUDENT

Weekly Logs

Teamwork

Problem and Solution research

Technical Research

Challenges

Jira and Git

Week 6

Thursday 26 February 2026 10:40

```
#Field mapping to normalize Outlook response keys to match Gmail's schema.
#This keeps the service layer untouched while giving the frontend a consistent API.
FIELD_REMAP = {
    'sender': 'from',
    'sender_name': 'from_name',
    'received_time': 'date',
}
```

```
def normalize(messages):
    normalized = []
    for msg in messages:
        normalized.append({FIELD_REMAP.get(k, k): v for k, v in msg.items()})
    return normalized
```

This week the first thing I do is the data normalization. The problem in the project is that for gmail and outlook, they have different names for similar stuff, for example:

1. In Gmail, we use "from" where Outlook is using "sender"
2. Gmail uses "date", Outlook uses "received_time"

The problem with this is that for example:

Gmail returns
{ "from": "john@gmail.com", "date": "Mon, 24 Feb 2026" }

Outlook returns
{ "sender": "john@outlook.com", "received_time": "2026-02-24T10:30:00Z" }

In the frontend, I will need seaprate logic for each service for example:

```
const senderEmail = email.from || email.sender;
const emailDate = email.date || email.received_time;
```

But with normalize, every field is unified, and so I can have something like this:

```
const senderEmail = email.from;
const emailDate = email.date;
```

Also, the changes is made in the router layer instead of in the service layer, so that the service layer stays true to the Microsoft Graph API naming.

https://en.wikipedia.org/wiki/Internet_Message_Access_Protocol

Week 4 standup

Tuesday 17 February 2026 10:12

Last Week: Finished application API layer, and start working on integrating LLM API key for AI Agent workflow.

This week: Complete set up LLM and start working on the RAG framework for user personalized and more accurate performance.

Blockers: None

David:

Last week? Trained the agent for 200 timesteps and then ran an evaluation script to see if it was improving and it was. Figured out the Cyborg network topology and started designing a simplified version for now in mininet.

This week? List & implement blue agent cyborg actions and the equivalent in mininet, finish designing the mininet topology, start designing scripted attack scenarios for the agent to defend against when i get to that stage

Blockers? Nada

Anthony:

Last week: calendar card and date logic for delivery schedules

This week: delivery cards for Home Screen

Blockers: none

Andrew:

Mathew:



Add Page

SEM 1

SEM 2

Week 1

Week 3

Week 4

Week 5

Week 6

Week 7

Weke 8

Week 9

Week 10

Week 11

Week 12

Week 13



Add Page

Week 2 standup

Week 3 standup

Week 4 standup

Week 5 standup

Week 6 standup

Week 7 standup

Week 8 standop

Week 9 standup

Week 10 standup

Week 11 standup

Week 12 standup

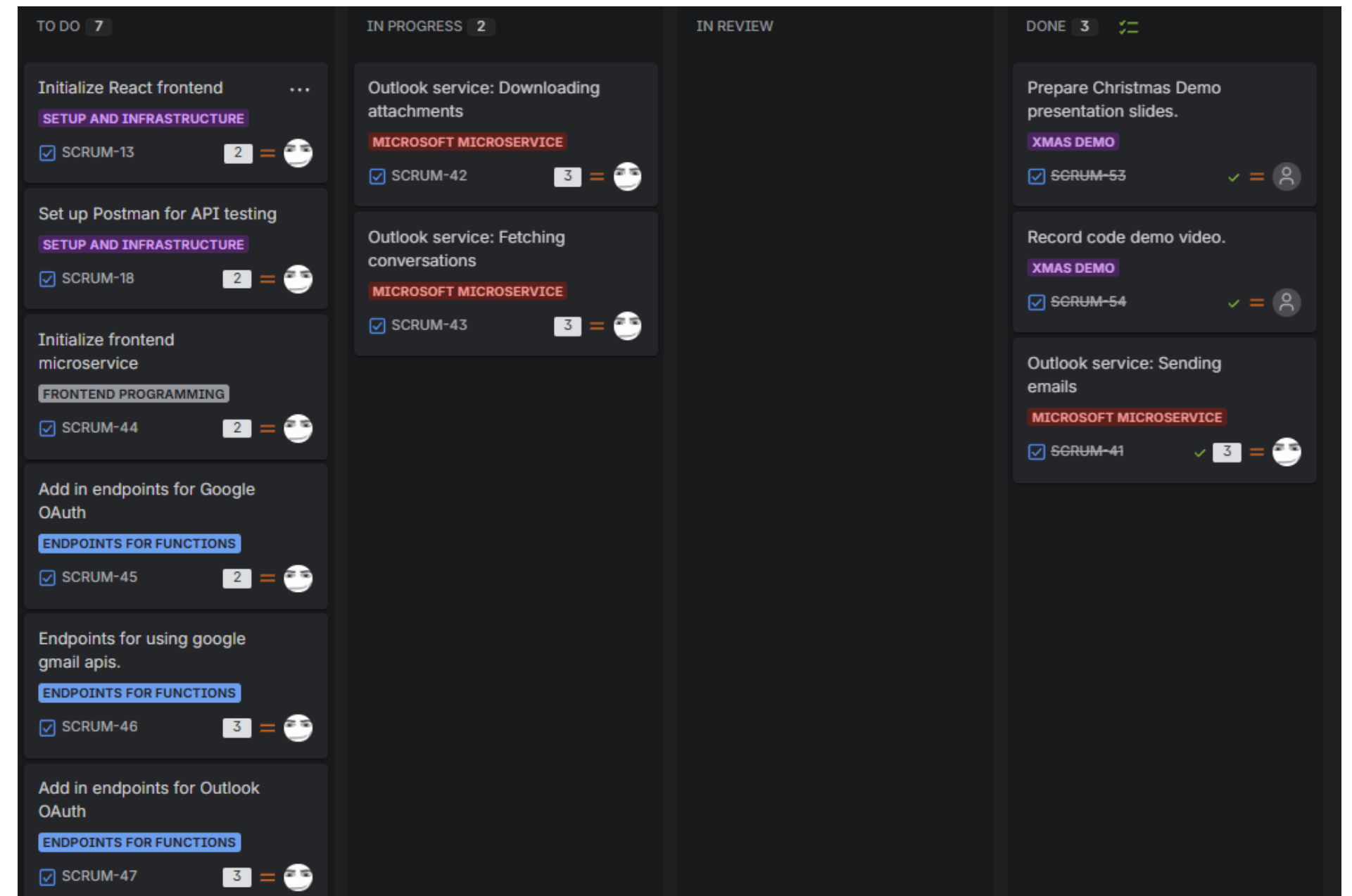
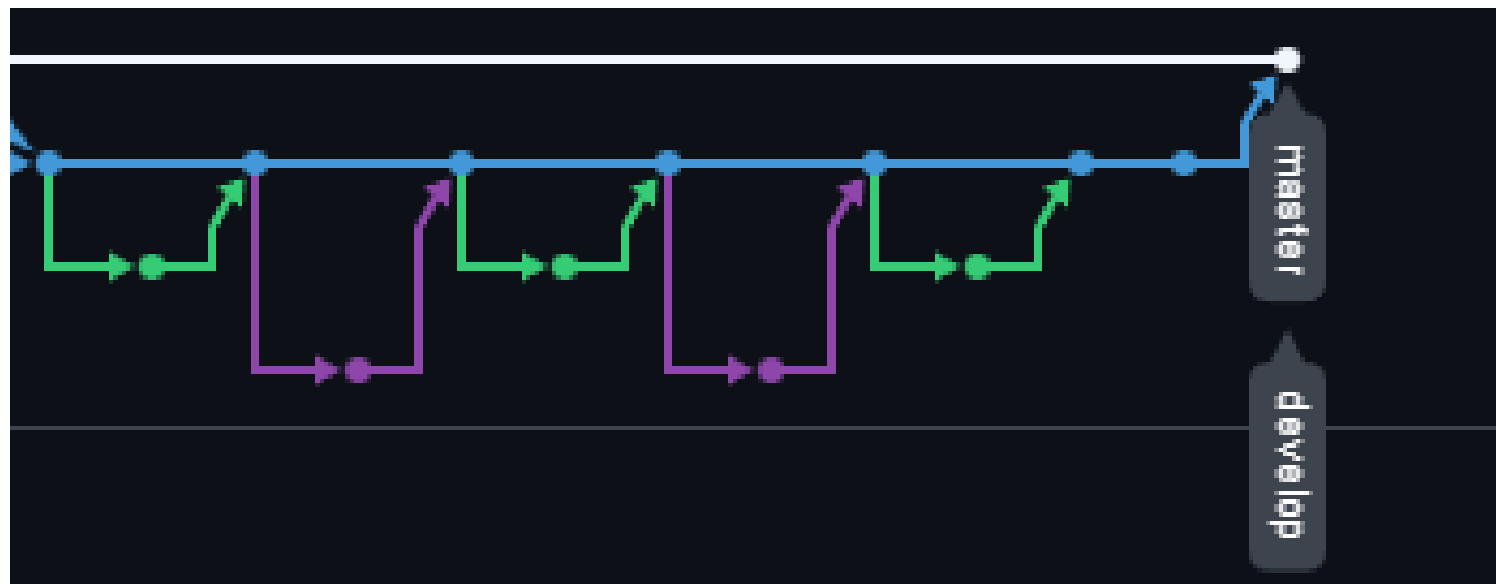
SEM 2

Week 1 standup

Week 2 standup

Week 3 standup

Week 4 standup



Problem Solving

◆ Time management

Use weekend and reading week as catchup.

◆ API Documentations

Found guides and summarized documentation on Internet, YouTube tutorial/crash courses. Uses AI to help summarize and explain some of the documentation.

◆ Underestimated research time

The time I spent on research is longer than I expected, which delays my project development, so I have to add in more tasks to each sprint to catch up the progress.

AI Use

◆ **Early stage project planning**

Uses AI to help generate an over project planning for reference.

◆ **API Documentation summarizing and explaining**

Here is where I use AI the most, which I use to summarize and explain the documents that I don't understand.

◆ **Comparing frameworks and tools**

I also used ChatGPT to help me compare different frameworks and tools so that I know which one suits best for my project.



Thank You

